STELLAR SCAFFOLD · HACKATHON QUICK REFERENCE

From idea to working dApp, *fast.*

Stellar Scaffold is a toolkit for full-stack Stellar apps. One command gives you Rust smart contracts, a React or Svelte frontend, a local network, and typed TypeScript clients that rebuild every time your contract changes.

THE AHA COMPANY

`scaffold.yml` = your dependencies

Your app's contract dependencies, per network: what they are and where to get them.

Network choice and secrets go in `app/.env`.

`app-lib/clients` = generated

A typed client per contract, per network, regenerated on every build and gitignored. Import by name; never paste an address.

Don't edit it: wrap clients under `app/`.

`npm run dev` = the loop

Runs `stellar scaffold watch` and Vite together. Save a contract and it rebuilds, redeploys on the local network, and regenerates its client.

App runs at localhost:5173 by default

1 Set up

You need **Rust + Cargo**, **Node.js + npm**, the **Stellar CLI** (`stellar`), and **Docker** for the local network. Scaffold works best with Stellar Registry (see Section 5); both install as plugins under `stellar scaffold` and `stellar registry`.

```
cargo install --locked stellar-scaffold-cli stellar-registry-cli # or: cargo binstall
stellar scaffold doctor  # checks your setup and suggests a fix for anything missing
```

2 Start a project

```
stellar scaffold init my-app  # asks: template + package manager
cd my-app
npm install  # or the package manager you picked
npm run dev  # http://localhost:5173
```

Skip the prompts: `--template react|svelte|none`, `-p <package-manager>`, `-y`.

```
my-app/
├── contracts/        Rust contracts → Wasm
├── app/              your frontend
│   └── .env          network + secrets
├── app-lib/          wallet, network, helpers
│   └── clients/      generated
└── scaffold.yml      contract dependencies
```

3 Add a contract

Start from a vetted example. `generate` writes the crate but does **not** add it to `scaffold.yml`; do that yourself.

```
stellar scaffold generate contract --ls  # list available examples
stellar scaffold generate contract --from oz/nft-royalties  # or stellar/hello-world
stellar scaffold generate contract --from-wizard  # open the OpenZeppelin Contract Wizard
```

```
# scaffold.yml
contracts:
  my-nft:                          # client name; exported as myNft
    type: workspace                # built from contracts/; deployed only where needed, e.g. local
    source: nft-royalties          # the crate's package name in its Cargo.toml
    args:                          # constructor args, by parameter name
      owner: ${account.me}
    networks:                      # networks where your app uses this contract
      local:
```

4 Call it from your app

Each contract becomes a typed, ready-to-call object bound to the current network and address. Switching networks needs no code change. Full pattern: the template's `GuessTheNumber` component.

```
import { guessTheNumber } from "@stellar-scaffold/app-lib/clients"

// Read: simulated, no signature needed
const { result } = await guessTheNumber.get_prize()

// Write: build the transaction, sign with the connected wallet, submit
const { address, signTransaction } = useWallet()
const tx = await guessTheNumber.guess({ a_number: 5n, guesser: address }, { publicKey: address })
const { result: won } = await tx.signAndSend({ signTransaction })
```

5 Go to testnet and ship

Scaffold only deploys locally. Put contracts on testnet with **Stellar Registry**, then point `scaffold.yml` at them.

a. In `app/.env`, set `STELLAR_NETWORK=testnet`. Check each contract's `[package.metadata.stellar]` in `Cargo.toml`: `cargo_inherit = true` copies `authors`, `homepage`, `repository`, and `version` (as `binver`) into the Wasm.

b. Compile, publish + deploy via Stellar Registry:

```
stellar scaffold build --compile-only  # testnet Wasm
stellar keys fund me --network testnet
stellar registry publish \
  --wasm path/to/nft_royalties.wasm \
  --wasm-name unverified/nft-royalties --binver 1.0.0
stellar registry deploy \
  --contract-name unverified/my-nft \
  --wasm-name unverified/nft-royalties --version 1.0.0 \
  -- --owner me  # constructor args
```

c. Point scaffold.yml at it for testnet:

```
contracts:
  my-nft:
    networks:
      local:            # built from contracts/
      testnet:
        type: contract  # already deployed: fetch it
        source: C...    # ID from registry (names soon)
```

d. Fetch, regenerate clients, build:

```
stellar scaffold config check --network testnet
stellar scaffold build --network testnet
npm run build  # static site in dist/
```

Contracts can differ between networks: make sure the frontend still typechecks against the fresh testnet clients. Host `dist/` anywhere static.

6 Command cheat sheet

All commands start with `stellar scaffold`. Add `--help` to any of them.

Command	What it does
`init <path>`	New project from a template (`--template`, `-p`, `-y`)
`generate contract`	Copy an example contract into `contracts/` (`--from`, `--ls`, `--from-wizard`)
`watch`	On change: rebuild, redeploy locally, regenerate clients (`npm run dev`)
`build [--network n]`	Resolve contracts for a network (deploy only if needed + allowed); make clients
`doctor [--strict] [--json]`	Diagnose toolchain, project, and network problems, with a fix for each
`config check | show`	Validate `scaffold.yml`, or print the resolved config for a network
`clean`	Wipe local/test deployments, generated clients, and aliases for a fresh deploy
`upgrade [path]`	Turn an existing Soroban workspace into a Scaffold project
`update-env --name N --value V`	Set a variable in `.env`

7 Handy extras

Teach your coding agent

Add the Stellar Scaffold skill from **skills.stellar.org** to Claude Code, Codex, or Cursor.

Find contracts to depend on

Browse published Wasms and contracts at **rgstry.xyz**; any of them can be a `scaffold.yml` dependency.

8 Common snags

Symptom	Fix
Changed constructor args, nothing happened	Only a Wasm change redeploys. Run `stellar scaffold clean`, then `npm run dev`.
My edits in app-lib/clients vanished	That folder is regenerated every build. Wrap the client in your own module under `app/`.
Contract missing from the clients import	It isn't listed under the current network in its `networks:` entry in `scaffold.yml`.
App breaks on testnet but works locally	Testnet contract differs. Rebuild with `--network testnet`; fix type errors.
Local network won't start	Start Docker; `stellar scaffold doctor` checks the daemon and RPC health.
Built for an unexpected network	Check `stellar scaffold config show`. A `STELLAR_NETWORK` set in your shell beats the one in `app/.env`.